UNIVERSIDAD TECNOLÓGICA DE GUTIÉRREZ ZAMORA

PROGRAMA EDUCATIVO:

Ingeniería en Desarrollo y Gestión de Software

NOMBRE DE LA MATERIA:

Desarrollo Móvil Integral

NOMBRE DE LA TAREA:

Unidad II — Seguridad MASVS

Auditoría básica de seguridad con OWASP MASVS / MASTG
(aplicación propia: TotoNinin)

ELABORADO POR:

Héctor Zahid Juárez Simbrón

DOCENTE:

Lothar Yamir Hernández Gaona

1010 - “AEI”

SEPTIEMBRE – DICIEMBRE 2026

Índice

Alcance y controles seleccionados.....3
Procedimiento de prueba.....5
Resultados y evidencias.....7
Hallazgos y nivel de riesgo.....21
Correcciones implementadas.....27
Repetición de pruebas y conclusiones.....29
Entrega y repositorio.....32

Alcance y controles seleccionados

Qué se auditó

TotoNinin es una aplicación para gestionar eventos. Los usuarios se registran, ven un calendario con categorías de eventos y pueden dejar comentarios, y un administrador maneja todo desde un panel. Está hecha en PHP 8 con MySQL y las páginas se arman en el servidor: el navegador solo recibe el resultado ya listo. No usa ningún framework, y mezcla mysqli procedural con mysqli orientado a objetos.

No es una app móvil. El ZIP del proyecto no trae ningún APK ni IPA (los archivos con los que se instalan las apps en Android y en iPhone), solo el código web. Por eso, en la aclaración de alcance explico cómo usé MASVS en este caso.

Como guía usé OWASP MASVS en su versión 2, que divide la seguridad de una app en ocho grupos: STORAGE (cómo se guardan los datos), CRYPTO (uso de cifrado y hashes), AUTH (inicio de sesión y permisos), NETWORK (comunicación por la red), PLATFORM (cómo se relaciona la app con el sistema), CODE (seguridad del código), RESILIENCE (qué tan difícil es modificar la app) y PRIVACY (manejo de datos personales). Para saber cómo probar cada punto me apoyé en la guía MASTG.

Trabajé con dos versiones. v1-original es el código tal como venía en el ZIP (lo recibí el 29/10/2026) y v2-corregida es el mismo código con las 6 correcciones de esta auditoría. Las dos están completas en el repositorio. Las pruebas las corrí con el servidor integrado de PHP (php -S). No pude instalar MySQL, así que usé una base de datos SQLite que responde igual a las llamadas mysqli que hace la aplicación (pruebas/mysqli_shim.php). Eso no cambia nada de la seguridad de la app, y además revisé cada hallazgo leyendo el código PHP real.

Reglas de trabajo

Solo probé mi propia aplicación y no ataqué ningún servicio externo. Todo se hizo con datos de prueba: usuarios inventados con correos @totoninin.test y claves de prueba (por ejemplo admin@totoninin.test y user@totoninin.test), sin información real de nadie. En las capturas, las claves, los hashes y las cookies de sesión están ocultos. Antes de corregir guardé una copia completa del código original (v1-original) para poder comparar.

Aclaración de alcance: MASVS aplicado a una aplicación web

MASVS está hecho para apps móviles que se instalan en el celular, y TotoNinin corre en un servidor, así que no todos los puntos aplican. Usé los grupos que sí tienen un equivalente en una aplicación web: STORAGE, CRYPTO, AUTH, NETWORK, CODE y PLATFORM (en PLATFORM solo el control de acceso a las pantallas de administrador y la revisión de los archivos que se suben). El material de la unidad también incluye puntos que no dependen de la plataforma, como "HTTP o TLS mal validado" y "Secretos incluidos en Git".

Dejé como No aplica lo que solo tiene sentido en una app instalada en el celular: RESILIENCE y, dentro de PLATFORM, los permisos del sistema, los WebViews y los enlaces profundos. La razón es que el código de TotoNinin nunca llega al dispositivo de quien ataca, porque se queda en el servidor. Cada caso está explicado en el checklist del final.

Controles seleccionados

Evalué STORAGE, CRYPTO, AUTH y CODE completos, y PLATFORM solo en parte. PRIVACY también lo evalué y la app cumple, con una observación menor que no bloquea nada. NETWORK y RESILIENCE quedaron como No aplica, con su justificación en el checklist.

Procedimiento de prueba

Inventario

Primero anoté qué maneja la app. En la base de datos hay cuatro tablas: users (nombre, correo y hash de contraseña), events (título, descripción, fecha, foto y tipo), comments (texto libre ligado a un evento) y event_type (catálogo de categorías). Las funciones delicadas son iniciar sesión, el registro público, el alta, edición y eliminación de usuarios, eventos y categorías desde el panel de administración, la subida de fotos, los comentarios y cerrar sesión.

En el código original hay dos niveles de permisos que no están escritos en ningún lado: usuario con sesión y administrador, y este último se decide comparando el correo con un valor fijo, sin tabla de roles. Sin iniciar sesión se puede entrar a index.php, paginas/registro.php, paginas/login.php, paginas/informacion.php y paginas/preguntas.php.

Revisión del código

Revisé a mano todo el código PHP (2,673 líneas en 37 archivos) buscando entradas del usuario pegadas dentro de consultas SQL, credenciales escritas en el código, el algoritmo con el que se guardan las contraseñas, la validación de los archivos subidos, la configuración de las cookies de sesión, el manejo de errores y si existía un .gitignore. Después de corregir comparé línea por línea v1-original contra v2-corregida para confirmar que cada cambio es pequeño y no toca nada que no tenga que ver con el hallazgo.

Pruebas con la aplicación funcionando

Levanté la aplicación con el servidor integrado de PHP sobre la base de datos de prueba y escribí un script de Python que hace las pruebas solo y se puede repetir las veces que haga falta (pruebas/run_tests.py, también en el repositorio). Para cada

versión del código, el script deja la base de datos en un estado conocido antes de cada caso para que las pruebas no se afecten entre sí y manda peticiones HTTP reales, con y sin sesión, con datos maliciosos y con datos normales. Guarda lo que pasó (código de estado, cabeceras importantes y partes de la respuesta) ocultando las cookies de sesión y los hashes. Al final compara el resultado con lo que se espera de cada hallazgo y marca el caso como SEGURO/CUMPLE o VULNERABLE/NO CUMPLE.

En total fueron 24 casos: 19 de seguridad, repartidos en los 6 hallazgos, y 5 de regresión (login normal, calendario, panel de administración, registro público y página de inicio), que sirven para comprobar que las correcciones no rompieran nada que ya funcionaba.

Cómo calculé el riesgo

A cada hallazgo le puse una probabilidad (qué tan fácil es que alguien lo aproveche) y un impacto (qué tanto daño haría), los dos del 1 al 3. El riesgo es la multiplicación de ambos: de 1 a 3 es Bajo, de 4 a 6 Medio, de 7 a 8 Alto y 9 Crítico. Es la escala que vimos en la unidad, no una escala oficial de OWASP.

Resultados y evidencias

En v1-original solo 8 de los 24 casos salieron bien: los 5 de regresión, que comprueban que la app funciona normal, y 3 de seguridad. Los otros 16 casos de seguridad fallaron. En v2-corregida pasaron los 24, así que los problemas se corrigieron sin romper nada de lo que ya funcionaba.

Evidencia por hallazgo

*Las claves, los hashes y las cookies de sesión están ocultos con **** en todas las capturas. En las capturas de código, las líneas resaltadas son las que importan para cada hallazgo, y el número de línea es el real del archivo.*

VULN-01 · Inyección SQL

En v1-original, scripts/login.php arma la consulta pegando directamente lo que escribe el usuario (líneas 21 a 23):

PHP login.php

v1-original › scripts › login.php

```
16  // Procesar formulario de login
17  if ($_SERVER["REQUEST_METHOD"] == "POST") {
18      $email = $_POST["email"];
19      $password = $_POST["pass"];
20
21      // Consultar usuario en la base de datos
22      $sql = "SELECT * FROM users WHERE email_us = '$email' AND password_us =
    '$password'";
23      $result = $conn->query($sql);
24
25      if ($result->num_rows > 0) {
26        // Usuario encontrado, iniciar sesión
27        session_start();
28        $_SESSION["logged_in"] = true;
29        $_SESSION["email"] = $email;
30        // Redirigir a la página de dashboard
31        header("Location: ../paginas/calendario.php");
32        exit;
33    } else {
```

v1-original PHP UTF-8 Espacios: 4

En v2-corregida, el login llama a la función authenticate() en lugar de armar el SQL a mano:

PHP login.php

v2-corregida › scripts › login.php

```php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $conn = db();
    $email = trim($_POST["email"] ?? '');
    $clave = $_POST["pass"] ?? '';

    $user = ($email !== '' && $clave !== '') ? authenticate($conn, $email, $clave) : null;

    if ($user) {
        login_user($user['email_us'], $user['Name_us']); // regenera el ID de sesion
        header("Location: ../paginas/calendario.php");
        exit;
    }
    http_response_code(401);
    header("Content-Type: application/json");
    echo json_encode(array("success" => false, "error" => "Credenciales incorrectas"));
    $conn->close();
    exit;
```

v2-corregida PHP UTF-8 Espacios: 4

Y esa función usa una consulta preparada, donde los datos viajan separados de la instrucción SQL (includes/auth.php):

PHP auth.php

v2-corregida › includes › auth.php

```php
function authenticate($conn, $correo, $clave) {
    $st = $conn->prepare('SELECT ID_us, Name_us, email_us, password_us FROM users WHERE email_us = ?');
    $st->bind_param('s', $correo);
    $st->execute();
    $fila = $st->get_result()->fetch_assoc();

```

v2-corregida PHP UTF-8 Espacios: 4

Resultado del ataque antes de corregir:

RESULTADO DE LA PRUEBA

VULN-01 · antes de corregir

```
Petición:  POST scripts/login.php
email =  x' OR '1'='1
pass  =  x' OR '1'='1
Respuesta:  302 Found, Location: ../paginas/calendario.php
Set-Cookie: PHPSESSID=****
✘ FALLA: entró sin credenciales válidas.
```

Pruebas NO CUMPLE

Resultado del mismo ataque después de corregir:

RESULTADO DE LA PRUEBA

VULN-01 · después de corregir

```
Petición:  POST scripts/login.php
email =  x' OR '1'='1
pass  =  x' OR '1'='1
Respuesta:  401 Unauthorized
{"success":false,"error":"Credenciales incorrectas"}
✔ OK: el texto se trata como texto normal y no se inicia sesión.
```

Pruebas CUMPLE

VULN-02 · Control de acceso roto

En v1-original, eliminar_usuario.php solo inicia la sesión, pero no revisa quién es el usuario ni qué rol tiene, así que cualquiera que llegue al archivo puede borrar (líneas 28 a 35):

PHP eliminar_usuario.php

v1-original › scripts › eliminar_usuario.php

```
1  <?php
2  session_start();
3
4  include "conexion.php";
⋮
28 if(!empty($_POST)){
29      $id_usuario = $_POST['id'];
30      $id_usuario = mysqli_real_escape_string($conection, $id_usuario);
31
32      $query_delete = mysqli_query($conection, "DELETE FROM users WHERE ID_us =
    $id_usuario");
33
34      if($query_delete){
35          header("location: lista_usuarios.php");
```

v1-original PHP UTF-8 Espacios: 4

En v2-corregida, el archivo empieza llamando a require_admin():

PHP eliminar_usuario.php

v2-corregida › scripts › eliminar_usuario.php

```
1  <?php
2  require_once __DIR__ . '/../includes/auth.php';
3  require_admin(); // VULN-02: solo administradores
4
5  include "conexion.php";
⋮
30 if(!empty($_POST)){
31      $id_usuario = $_POST['id'];
32      $id_usuario = (int)$id_usuario;
33
34      $query_delete = mysqli_query($conection, "DELETE FROM users WHERE ID_us =
    $id_usuario");
35
36      if($query_delete){
```

v2-corregida PHP UTF-8 Espacios: 4

Esa función exige que haya sesión iniciada y que el usuario sea administrador (includes/auth.php):

PHP auth.php

v2-corregida › includes › auth.php

```php
/** Exige sesión iniciada Y rol administrador (control de acceso en el servidor). */
function require_admin() {
    require_login();
    if (empty($_SESSION['es_admin'])) {
        http_response_code(403);
        exit('Acceso denegado');
    }
}
```

v2-corregida PHP UTF-8 Espacios: 4

Resultado de la prueba sin iniciar sesión, antes de corregir:

RESULTADO DE LA PRUEBA

VULN-02 · antes de corregir

```
Petición:  POST scripts/eliminar_usuario.php?id=3 (sin iniciar sesión)
Respuesta:  302 Found, Location: lista_usuarios.php
✘ FALLA: el usuario con id=3 se eliminó sin que nadie hubiera iniciado sesión.
```

Pruebas NO CUMPLE

Resultado de la misma prueba después de corregir:

RESULTADO DE LA PRUEBA

VULN-02 · después de corregir

```
Petición:  POST scripts/eliminar_usuario.php?id=3 (sin iniciar sesión)
Respuesta:  302 Found, Location: ../paginas/login.php
✔ OK: lo manda al login y el usuario sigue en la base de datos.
```

Pruebas CUMPLE

VULN-03 · Contraseñas guardadas con MD5

En v1-original la contraseña se guarda con md5() (línea 17):

PHP registro_usuario.php

v1-original › scripts › registro_usuario.php

```
15  $Name_us = $_POST['Name_us'];
16  $email_us  = $_POST['email_us'];
17  $password_us  = md5($_POST['password_us']);
⋮
27      $query_insert = mysqli_query($conection,"INSERT INTO
    users(Name_us,email_us,password_us)
28
    VALUES('$Name_us','$email_us','$password_us')");
```

v1-original PHP UTF-8 Espacios: 4

En v2-corregida se usa password_hash() (línea 18) y los datos van en una consulta preparada:

PHP registro_usuario.php

v2-corregida › scripts › registro_usuario.php

```
16  $Name_us = trim($_POST['Name_us']);
17  $email_us = trim($_POST['email_us']);
18  $password_us = password_hash($_POST['password_us'], PASSWORD_DEFAULT); //
    VULN-03: bcrypt con sal (antes md5)
⋮
27      $ins = $conection->prepare("INSERT INTO
    users(Name_us,email_us,password_us) VALUES(?,?,?)");
28      $ins->bind_param("sss", $Name_us, $email_us, $password_us);
```

v2-corregida PHP UTF-8 Espacios: 4

Además, al iniciar sesión, si la cuenta todavía tiene un hash MD5 viejo se acepta una sola vez y se cambia por bcrypt (includes/auth.php):

PHP auth.php

v2-corregida › includes › auth.php

```php
$hash = $fila['password_us'];
$ok = false;
$migrar = false;
if (password_verify($clave, $hash)) {
    $ok = true;
    $migrar = password_needs_rehash($hash, PASSWORD_DEFAULT);
} elseif (preg_match('/^[a-f0-9]{32}$/i', $hash) &&
hash_equals(strtolower($hash), md5($clave))) {
    $ok = true;
    $migrar = true;
}
```

v2-corregida PHP UTF-8 Espacios: 4

Lo que se guardaba en la base de datos antes de corregir:

RESULTADO DE LA PRUEBA

VULN-03 · antes de corregir

```
Consulta:  SELECT password_us FROM users WHERE email_us = 'nuevo@totoninin.test'
Resultado:  [hash MD5 de 32 caracteres, oculto]
✘ FALLA: MD5 sin sal, se puede descifrar con un diccionario en segundos.
```

Pruebas NO CUMPLE

Lo que se guarda después de corregir:

RESULTADO DE LA PRUEBA

VULN-03 · después de corregir

```
Consulta:  SELECT password_us FROM users WHERE email_us = 'nuevo@totoninin.test'
Resultado:  [hash bcrypt $2y$10$..., oculto]
✔ OK: bcrypt usa sal y es lento de adivinar, no se descifra con un diccionario.
```

Pruebas CUMPLE

VULN-04 · Subida de archivos sin validar

En v1-original, registro_evento.php guarda la foto con el nombre que manda el navegador y sin revisar qué contiene (líneas 45 a 48):

PHP registro_evento.php

v1-original › scripts › registro_evento.php

```
40  if ($foto['error'] !== 0) {
41    echo "Error: Foto inválida";
42    exit;
43  }
44
45  // Mover la foto a una carpeta temporal
46  $rutaTemporal = $foto['tmp_name'];
47  $rutaDestino = '../fotos/' . $foto['name'];
48  move_uploaded_file($rutaTemporal, $rutaDestino);
```

v1-original PHP UTF-8 Espacios: 4

En v2-corregida, includes/upload.php revisa que sea una subida real, que no pese más de 2 MB, que su tipo real sea de imagen y que se pueda leer como imagen. Después la guarda con un nombre aleatorio:

PHP upload.php

v2-corregida › includes › upload.php

```php
function guardar_foto_evento($f) {
    if (!is_array($f) || ($f['error'] ?? UPLOAD_ERR_NO_FILE) !== UPLOAD_ERR_OK) {
        return null;
    }
    if (!is_uploaded_file($f['tmp_name']) || $f['size'] > 2 * 1024 * 1024) {
        return null;
    }
    $permitidos = ['image/jpeg' => 'jpg', 'image/png' => 'png', 'image/gif' => 'gif', 'image/webp' => 'webp'];
    $mime = (new finfo(FILEINFO_MIME_TYPE))->file($f['tmp_name']);
    if (!isset($permitidos[$mime]) || @getimagesize($f['tmp_name']) === false) {
        return null;
    }
    $nombre = bin2hex(random_bytes(16)) . '.' . $permitidos[$mime];
    if (!move_uploaded_file($f['tmp_name'], __DIR__ . '/../fotos/' . $nombre)) {
        return null;
    }
    return '../fotos/' . $nombre;
}
```

v2-corregida PHP UTF-8 Espacios: 4

Además, la carpeta fotos/ tiene un archivo .htaccess que impide ejecutar PHP ahí:

CFG .htaccess

v2-corregida › fotos › .htaccess

```
<FilesMatch "\.(php|phtml|phar|php[0-9])$">
    Require all denied
</FilesMatch>
php_flag engine off
Options -ExecCGI -Indexes
```

v2-corregida Apache UTF-8 Espacios: 4

Resultado de subir shell.php antes de corregir:

RESULTADO DE LA PRUEBA

VULN-04 · antes de corregir

```
Petición 1:  POST registro_evento.php, foto = shell.php (tipo declarado:
image/jpeg)
Petición 2:  GET fotos/shell.php
Respuesta:  RCE_42
✘ FALLA: el archivo .php subido se ejecutó en el servidor.
```

Pruebas NO CUMPLE

Resultado de subir shell.php después de corregir:

RESULTADO DE LA PRUEBA

VULN-04 · después de corregir

```
Petición:  POST registro_evento.php, foto = shell.php (tipo declarado: image/jpeg)
Respuesta:  400 Bad Request, "Error: archivo de imagen no válido"
✔ OK: se rechaza porque su contenido real no es una imagen y no se guarda.
```

Pruebas CUMPLE

VULN-05 · Credenciales escritas en el código

En v1-original, scripts/conexion.php trae el usuario root y la contraseña vacía escritos directo (líneas 3 a 6):

PHP conexion.php

v1-original › scripts › conexion.php

```php
<?php

    $host = 'localhost';
    $user = 'root';
    $password = '';
    $db = 'totoninin';

    $conection = @mysqli_connect($host,$user,$password,$db);

    if(!$conection){
        echo "Error en la conexión";
    }
```

v1-original PHP UTF-8 Espacios: 4

En v2-corregida, conexion.php ya no tiene datos de acceso y le pide la conexión a db():

PHP conexion.php

v2-corregida › scripts › conexion.php

```php
<?php
// VULN-05: las credenciales ya no estan en el codigo (ver includes/config.php)
require_once __DIR__ . '/../includes/db.php';
$conection = db();
```

v2-corregida PHP UTF-8 Espacios: 4

Los datos se leen de variables de entorno o de config.local.php, que no se sube a Git (includes/config.php):

PHP config.php

v2-corregida › includes › config.php

```php
function cfg($clave, $defecto = null) {
    static $local = null;
    if ($local === null) {
        $archivo = __DIR__ . '/config.local.php';
        $local = is_file($archivo) ? (require $archivo) : [];
    }
    $valor = getenv($clave);
    if ($valor !== false && $valor !== '') {
        return $valor;
    }
    return $local[$clave] ?? $defecto;
}
```

v2-corregida PHP UTF-8 Espacios: 4

Resultado de buscar las credenciales en el código antes de corregir:

RESULTADO DE LA PRUEBA

VULN-05 · antes de corregir

```
Búsqueda en el código:  new mysqli("localhost", "root"
Archivos con coincidencias:  14
✘ FALLA: usuario root y clave vacía escritos en 14 archivos.
```

Pruebas NO CUMPLE

Resultado de la misma búsqueda después de corregir:

RESULTADO DE LA PRUEBA

VULN-05 · después de corregir

```
Búsqueda en el código:  new mysqli("localhost", "root"
Archivos con coincidencias:  0
.gitignore incluye:  includes/config.local.php
✔ OK: no quedan credenciales en el código y el archivo con secretos no se sube a Git.
```

Pruebas CUMPLE

VULN-06 · Manejo inseguro de la sesión

En v1-original, el login inicia la sesión sin cuidar la cookie ni cambiar el ID de sesión (líneas 27 a 29):

PHP login.php

v1-original › scripts › login.php

```
25  if ($result->num_rows > 0) {
26    // Usuario encontrado, iniciar sesión
27    session_start();
28    $_SESSION["logged_in"] = true;
29    $_SESSION["email"] = $email;
30    // Redirigir a la página de dashboard
31    header("Location: ../paginas/calendario.php");
32    exit;
```

v1-original PHP UTF-8 Espacios: 4

En v2-corregida, includes/session.php configura la cookie con HttpOnly y SameSite (líneas 14 a 19), y login_user() cambia el ID de sesión (línea 41):

PHP session.php

v2-corregida › includes › session.php

```
11      $https = !empty($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off';
12      ini_set('session.use_strict_mode', '1');   // rechaza IDs de sesión no
    emitidos por el servidor
13      ini_set('session.use_only_cookies', '1');
14      session_set_cookie_params([
15          'lifetime' => 0,
16          'path'     => '/',
17          'secure'   => $https,
18          'httponly' => true,
19          'samesite' => 'Lax',
20      ]);
21      session_start();
 :
38  /** Marca la sesión como autenticada y renueva el ID (evita fijación de
    sesión). */
39  function login_user($correo, $nombre) {
40      secure_session_start();
41      session_regenerate_id(true);
42      $admin = cfg('ADMIN_EMAIL', '');
43      $_SESSION['correo'] = $correo;
44      $_SESSION['nombre'] = $nombre;
45      $_SESSION['es_admin'] = ($admin !== '' && strcasecmp($correo, $admin) ===
    0);
46      $_SESSION['ultimo_acceso'] = time();
47  }
```

v2-corregida PHP UTF-8 Espacios: 4

Resultado del inicio de sesión antes de corregir:

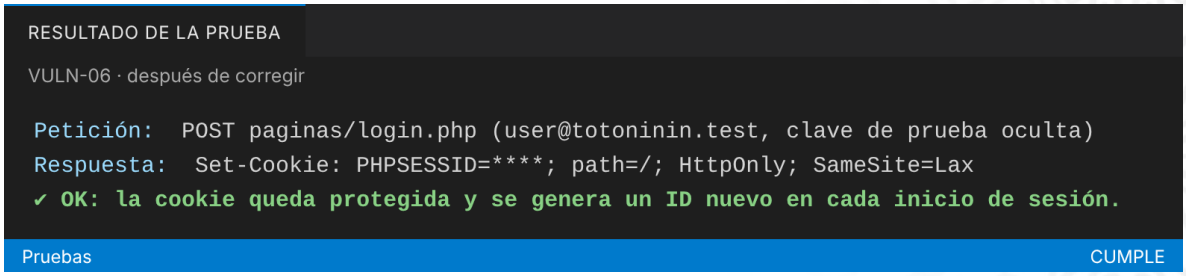
RESULTADO DE LA PRUEBA

VULN-06 · antes de corregir

```
Petición:  POST paginas/login.php (user@totoninin.test, clave de prueba oculta)
Respuesta:  Set-Cookie: PHPSESSID=****; path=/
✘ FALLA: la cookie no tiene HttpOnly ni SameSite y el ID de sesión no cambia al
entrar.
```

Pruebas NO CUMPLE

Resultado del inicio de sesión después de corregir:



Hallazgos y nivel de riesgo

VULN-01 — Inyección SQL

La inyección SQL pasa cuando el programa arma una consulta a la base de datos pegando directamente lo que escribió el usuario. Si alguien escribe código SQL en lugar de un correo, la base de datos lo ejecuta como si fuera parte de la consulta. Lo ubiqué en MASVS-CODE y, en segundo plano, MASVS-AUTH. Está en el login (scripts/login.php), en el parámetro "mes" de scripts/load_events.php y scripts/obtener_eventos.php, en el parámetro "pagina" de lista_usuarios.php, lista_eventos.php y lista_categoria.php, y en los formularios de registro_usuario.php, editar_usuario.php, registro_categoria.php y editar_categoria.php (correo y descripción).

Para comprobarlo mandé un POST a scripts/login.php con x' OR '1'='1 tanto en el correo como en la contraseña. Como la condición siempre sale verdadera, la app me dejó entrar sin conocer ninguna contraseña. Ya dentro con cualquier usuario, abrí scripts/load_events.php?mes=0 UNION SELECT ID_us,Name_us,password_us,... FROM users -- y esa página, que solo debería listar los eventos del mes, me mostró los hashes de contraseña de todos los usuarios.

Le puse probabilidad 3 (alta) e impacto 3 (alto), así que el riesgo es Crítico (9 de 9). Cualquiera puede entrar sin contraseña y leer los datos de todos los usuarios, y es muy fácil de hacer.

Cambié todas las consultas que armaban el SQL con lo que escribe el usuario por consultas preparadas (prepare y bind_param de mysqli). En una consulta preparada, la instrucción SQL y los datos viajan por separado, así que lo que escriba el usuario nunca se interpreta como código. Además, los valores que deben ser números, como mes, pagina e id, ahora se convierten a entero antes de usarlos. El login quedó en una sola función, authenticate(), dentro de includes/auth.php.

VULN-02 — Control de acceso roto en las funciones de administrador

Control de acceso roto quiere decir que la aplicación no revisa si quien pide una acción tiene permiso para hacerla. Las pantallas y acciones de administrador estaban abiertas para cualquiera. Lo ubiqué en MASVS-AUTH y, en segundo plano, MASVS-PLATFORM. Afecta a scripts/lista_usuarios.php, eliminar_usuario.php, editar_usuario.php, registro_usuario.php, lista_categoria.php, eliminar_categoria.php, registro_categoria.php, editar_categoria.php, lista_eventos.php, eliminar_confirmar_evento.php, registro_evento.php, editar_evento.php y paginas/admin.php.

Sin iniciar sesión pedí scripts/lista_usuarios.php y me mostró la lista completa de usuarios con nombre y correo. Con un POST a scripts/eliminar_usuario.php?id=3, también sin sesión, el usuario se borró de la base de datos. Con otro POST a scripts/editar_usuario.php?id=2 le cambié la contraseña a otro usuario y después pude entrar a su cuenta con la clave que puse yo. Por último, entré con un usuario normal (no administrador) y el panel de administración se abrió igual, porque ningún script revisa el rol.

Le puse probabilidad 3 (alta) e impacto 3 (alto), así que el riesgo es Crítico (9 de 9). Cualquier persona, incluso sin cuenta, puede ver, borrar y modificar usuarios, y hasta quedarse con la cuenta de otro.

Creé includes/auth.php con las funciones require_login() y require_admin(). Los 14 scripts de administración ahora piden sesión iniciada y rol de administrador antes de leer o escribir cualquier dato. El rol (es_admin) se calcula en el servidor a partir de ADMIN_EMAIL, así que el usuario no puede cambiarlo desde el navegador.

VULN-03 — Contraseñas guardadas con MD5

Una base de datos no debe guardar la contraseña tal cual, sino un hash, que es el resultado de revolverla con una fórmula que no se puede deshacer. MD5 es una fórmula vieja y muy rápida, y como se usó sin sal (un dato aleatorio que se mezcla

con la contraseña), es fácil adivinar contraseñas probando palabras de un diccionario. Lo ubiqué en MASVS-CRYPTO y, en segundo plano, MASVS-STORAGE. Está en scripts/registro.php y scripts/registro_usuario.php (alta de usuarios desde el panel de administración).

Registré un usuario desde scripts/registro_usuario.php y revisé la columna password_us en la base de datos. El valor era un hash MD5 de 32 caracteres hexadecimales y sin sal, que se puede descifrar en segundos si la base de datos llegara a filtrarse. El formulario público de registro (paginas/registro.php) ya usaba password_hash() con bcrypt desde el original, así que el problema estaba solo en el alta que hace el administrador y en la clase Registro, que quedó de una versión anterior.

Le puse probabilidad 2 (media) e impacto 2 (medio), así que el riesgo es Medio (4 de 9). Solo importa si alguien llega a ver la base de datos, y el registro público ya usaba bcrypt.

Cambié md5() por password_hash($clave, PASSWORD_DEFAULT), que usa bcrypt (un método más lento y con sal), en todos los lugares donde se crea o se edita una contraseña. También agregué una migración: si una cuenta vieja todavía tiene hash MD5, en su primer login correcto se valida una vez y se guarda de nuevo con bcrypt.

VULN-04 — Subida de archivos sin validar

Al crear un evento se puede subir una foto, pero la aplicación no comprueba que de verdad sea una imagen. Si alguien sube un archivo PHP, el servidor lo puede ejecutar como código, lo que se conoce como ejecución remota de código. Lo ubiqué en MASVS-CODE y, en segundo plano, MASVS-PLATFORM. Está en scripts/registro_evento.php y scripts/editar_evento.php (campo "foto").

Con una cuenta normal, creada desde el registro público, creé un evento y en la foto subí un archivo shell.php que contiene <?php echo 'RCE_'.(6*7); ?>. La aplicación

solo revisa $foto['error'] y mueve el archivo a la carpeta fotos/ con el mismo nombre y extensión que manda el navegador, sin comprobar qué contiene en realidad. Al abrir fotos/shell.php directamente, el servidor lo ejecutó como PHP y respondió RCE_42, o sea que cualquiera podría correr código en el servidor con los permisos del servidor web.

Le puse probabilidad 3 (alta) e impacto 3 (alto), así que el riesgo es Crítico (9 de 9). Con una cuenta normal se puede ejecutar código propio en el servidor.

Agregué includes/upload.php con la función guardar_foto_evento(). Revisa que el archivo venga de una subida real (is_uploaded_file), limita el tamaño, comprueba el tipo real con fileinfo en lugar del que dice el navegador y lo compara con una lista de imágenes permitidas. Después confirma con getimagesize() que de verdad sea una imagen y la guarda con un nombre aleatorio y la extensión que corresponde a su tipo, nunca la que mandó el cliente. Como protección extra puse un fotos/.htaccess que apaga la ejecución de PHP en esa carpeta.

VULN-05 — Credenciales escritas dentro del código

Dejar usuarios y contraseñas escritos dentro del código es un problema porque cualquiera que vea el código (por ejemplo, si se sube a GitHub) ya tiene esos datos. Los mensajes de error con demasiado detalle también le dan pistas a un atacante. Lo ubiqué en MASVS-STORAGE y, en segundo plano, MASVS-CODE. Aparece en scripts/conexion.php, bd_php/conexion.php, scripts/login.php, scripts/add_comment.php, scripts/evento_detalle.php, scripts/registro_evento.php, scripts/editar_evento.php, paginas/login.php, paginas/registro.php, index.php, scripts/load_events.php, scripts/obtener_eventos.php, scripts/eventos.php, scripts/registro.php y bd_php/usuarioBD.php.

Busqué en el código la cadena new mysqli("localhost","root","",...) y aparece repetida en 14 archivos, con el usuario root y la contraseña vacía escritos directo. Además bd_php/usuarioBD.php tenía error_reporting(E_ALL) e

ini_set("display_errors",1), y mostraba el mensaje de error de MySQL al usuario con die("Error de conexión: " . $e->getMessage()), lo que deja ver rutas, nombres de tablas y la versión de MySQL a cualquiera que visite la página. El proyecto tampoco tenía archivo .gitignore, así que si algún día se creaba un archivo de configuración con credenciales reales, se habría subido al repositorio sin ningún control.

Le puse probabilidad 2 (media) e impacto 2 (medio), así que el riesgo es Medio (4 de 9). Con root y sin contraseña quien llegue a la base de datos tiene control total, pero para eso primero necesita acceso al código o al servidor.

Dejé la conexión en includes/db.php e includes/config.php, que leen el host, el usuario, la clave y la base de datos de variables de entorno o del archivo includes/config.local.php, que no se sube al repositorio. Los errores de conexión y de consulta ahora se guardan con error_log() y al usuario solo le sale un mensaje general. También agregué un .gitignore que excluye config.local.php, los archivos *.env y los registros temporales de las pruebas.

VULN-06 — Manejo inseguro de la sesión

La sesión es lo que mantiene a un usuario dentro de la aplicación después de iniciar sesión, y se guarda en una cookie del navegador. Si no se cuida, esa cookie se puede robar o reutilizar. Lo ubiqué en MASVS-AUTH. Está en scripts/login.php, paginas/login.php, scripts/cerrar_sesion.php, includes/header.php e includes/nav.php.

Al iniciar sesión, la cabecera Set-Cookie del PHPSESSID no trae HttpOnly ni SameSite, y el identificador de sesión es el mismo que tenía el visitante antes de entrar porque no se llama a session_regenerate_id(). Eso permite que un atacante le dé a la víctima un identificador que él ya conoce, por ejemplo con un enlace, y lo siga usando cuando la víctima inicie sesión (fijación de sesión). Otro problema es que el login responde distinto cuando el correo no existe y cuando la contraseña es incorrecta, así que se pueden ir probando correos para saber cuáles están

registrados. Además, scripts/cerrar_sesion.php llama a session_destroy() pero nunca borra la cookie del navegador, por lo que el identificador sigue viajando en cada petición. Por último, el rol de administrador se decide comparando el correo con un valor fijo dentro de includes/nav.php, repetido en varios archivos, en lugar de calcularlo una sola vez en el servidor al autenticar.

Le puse probabilidad 2 (media) e impacto 2 (medio), así que el riesgo es Medio (4 de 9). Cada problema por separado necesita condiciones extra, como engañar a la víctima o tener acceso a su navegador.

Creé includes/session.php. La cookie de sesión ahora lleva HttpOnly (el código de la página no puede leerla) y SameSite=Lax (no se manda desde otros sitios), y Secure cuando se usa HTTPS; la sesión expira a los 30 minutos sin actividad; se llama a session_regenerate_id(true) en cada login; y logout_user() vacía $_SESSION, destruye la sesión en el servidor y expira la cookie en el navegador. El login ahora siempre responde "Credenciales incorrectas" sin decir cuál fue el motivo, y el rol es_admin se calcula una sola vez en el servidor al autenticar.

Correcciones implementadas

Archivos nuevos

Para no repetir código en cada archivo, creé varias piezas reutilizables en la carpeta includes/. includes/config.php lee la configuración (host, usuario y clave de la base de datos, y el correo del administrador) de variables de entorno o de includes/config.local.php, en lugar de tenerla escrita en el código, e includes/config.local.example.php es un ejemplo para que cada entorno cree su propio archivo, porque el verdadero no se sube a Git. includes/db.php tiene la función db(), que devuelve la conexión ya configurada; si algo falla, guarda el detalle en el registro del servidor y al usuario le muestra un mensaje general (VULN-05).

includes/session.php maneja la sesión de forma segura: secure_session_start() inicia la sesión con la cookie protegida y la expiración por inactividad, login_user() cambia el ID de sesión al entrar y logout_user() destruye la sesión y expira la cookie (VULN-06). includes/auth.php tiene require_login() y require_admin() (VULN-02) y authenticate(), que usa una consulta preparada, password_verify() y la migración de hashes MD5 viejos a bcrypt (VULN-01 y VULN-03). includes/upload.php tiene guardar_foto_evento(), que revisa el tipo real, el tamaño y que sea una imagen válida, y genera un nombre aleatorio (VULN-04).

Fuera de includes/ agregué fotos/.htaccess, que apaga la ejecución de PHP dentro de la carpeta de fotos subidas como protección extra para VULN-04, y un .gitignore que evita subir a Git includes/config.local.php, los archivos *.env y los archivos temporales de prueba (VULN-05).

Archivos modificados

En scripts/login.php, paginas/login.php y paginas/login.js corregí VULN-01 con la consulta preparada y VULN-06 con la sesión segura, el mensaje general de error y

textContent en lugar de innerHTML. En scripts/conexion.php, bd_php/conexion.php, index.php, bd_php/usuarioBD.php, paginas/registro.php y los scripts load_events, obtener_eventos, evento_detalle, add_comment, eventos y registro saqué las credenciales del código y dejé de mostrarle al usuario los detalles de los errores (VULN-05).

En los scripts de administración (registro_usuario, editar_usuario, registro_categoria, editar_categoria, eliminar_usuario, eliminar_categoria, eliminar_confirmar_evento, lista_usuarios, lista_categoria, lista_eventos y buscar_usuario, además de paginas/admin.php) agregué require_admin() para VULN-02 y cambié las consultas por consultas preparadas, con los parámetros numéricos convertidos a int, para VULN-01. scripts/registro_evento.php y scripts/editar_evento.php ahora validan la foto con includes/upload.php (VULN-04). Por último, scripts/cerrar_sesion.php, includes/header.php e includes/nav.php usan logout_user() y el rol es_admin calculado en el servidor (VULN-06).

El detalle línea por línea se puede ver en el repositorio, en el commit "Correcciones VULN-01..06 (OWASP MASVS) + evidencias + reporte técnico".

Repetición de pruebas y conclusiones

Pruebas después de corregir

Volví a correr las mismas 24 pruebas contra v2-corregida y 24 de 24 casos salieron SEGURO/CUMPLE, incluidos los 5 de regresión (login normal, calendario de eventos, panel de administración, registro público y página de inicio). Con esto se ve que las correcciones quitan los 6 hallazgos sin romper ninguna función de la aplicación.

Pendientes y recomendaciones

Quedaron cosas que no corregí. NETWORK no se pudo evaluar porque el proyecto no trae la configuración del servidor de producción; ahí habría que forzar HTTPS para que la comunicación vaya cifrada. Tampoco puse un límite de intentos de login, que ayudaría contra quien prueba contraseñas una por una, aunque ya estén guardadas con bcrypt.

De las cabeceras de seguridad solo agregué X-Content-Type-Options, X-Frame-Options y Referrer-Policy, y no puse Content-Security-Policy para no romper las fuentes e íconos externos que usa la app. La observación de PRIVACY (falta un aviso de privacidad y una opción para que el usuario borre su cuenta) la dejé como mejora futura. Por último, en un servidor real la base de datos debe usar un usuario de MySQL con permisos mínimos y una contraseña fuerte, nunca root sin clave; includes/config.local.example.php ya muestra ese cambio.

Conclusiones

Encontré 6 fallas, una más que el mínimo de 5 que pedía la tarea: 3 de riesgo Crítico (inyección SQL, control de acceso roto y subida de archivos sin validar) y 3 de riesgo Medio (MD5 en las contraseñas, credenciales escritas en el código y manejo inseguro de la sesión). Las seis quedaron corregidas y las comprobé con pruebas

que se pueden repetir, antes y después del cambio. Las 5 pruebas de regresión confirman que el inicio de sesión, el calendario, el panel de administración y el registro público siguen funcionando igual que antes.

RESILIENCE y la parte de PLATFORM que depende de permisos del sistema los dejé como No aplica porque TotoNinin es una aplicación web sin empaquetado móvil. La justificación está en el checklist de abajo.

Checklist MASVS

Área	Qué se revisó	Estado	Evidencia o justificación
STORAGE	Credenciales escritas en el código, manejo de secretos, errores con demasiado detalle y secretos en Git.	NC	VULN-05. Corregido y comprobado al volver a probar.
CRYPTO	Algoritmo de hash de las contraseñas y nombres de archivo o identificadores.	NC	VULN-03: de MD5 a bcrypt. Corregido.
AUTH	Inyección SQL en el login, acceso a las funciones de administrador, cookie de sesión, fijación de sesión, saber qué correos existen y cierre de sesión.	NC	VULN-01, VULN-02 y VULN-06. Corregidos.
NETWORK	Que la comunicación entre el navegador y el servidor esté protegida (HTTP/TLS).	N/A	El proyecto no incluye la configuración del servidor web ni un certificado de producción. Tampoco encontré en el código direcciones propias con http://, solo recursos externos como fuentes e íconos. Por eso no se puede evaluar con el código; en producción habría que forzar HTTPS.
PLATFORM	Validación de archivos subidos y control de acceso a las partes de administración.	NC	VULN-04 y VULN-02. Corregidos. No aplican los permisos del sistema, los WebViews (páginas web dentro de una app) ni los enlaces profundos, porque la app es web, se arma en el servidor y no tiene APK/IPA.
CODE	Validación de entradas, manejo de errores y modo de depuración.	NC	VULN-01 y VULN-05. Corregidos.

Área	Qué se revisó	Estado	Evidencia o justificación
RESILIENCE	Que sea difícil modificar la app o descompilarla.	N/A	No aplica. La app no se distribuye como archivo instalable (APK/IPA) sino como código PHP que corre en el servidor, así que no hay nada en el celular que alguien pueda descompilar o modificar.
PRIVACY	Pedir solo los datos necesarios, consentimiento, rastreadores y cuánto tiempo se guardan los datos.	C	Los datos que pide la app (nombre, correo, contraseña y comentarios) son los necesarios para funcionar y no encontré scripts de rastreo de terceros. Una observación menor: no hay aviso de privacidad ni forma de que el usuario elimine su propia cuenta; queda como mejora para después.

C = cumple · NC = no cumplía (se corrigió, se volvió a probar y ahora cumple) · N/A = no aplica (justificado en la tabla)

Entrega y repositorio

Subí todo al repositorio de GitHub https://github.com/hectorjua28/Seguridad-Practica-. Tiene la carpeta v1-original/ con la aplicación tal como la recibí, v2-corregida/ con las 6 correcciones, pruebas/ con run_tests.py, la capa de compatibilidad mysqli_shim.php y los resultados de cada ejecución (resultados.txt y resultados.json) de las dos versiones, evidencias/ con las capturas de este reporte y reporte/ con este documento.

El historial de Git empieza con un commit que guarda v1-original sin cambios y sigue con el commit de las correcciones VULN-01..06 y las evidencias, así que se puede comparar el antes y el después.